

Cornell Bowers C-IS

College of Computing and Information Science

Deep Learning

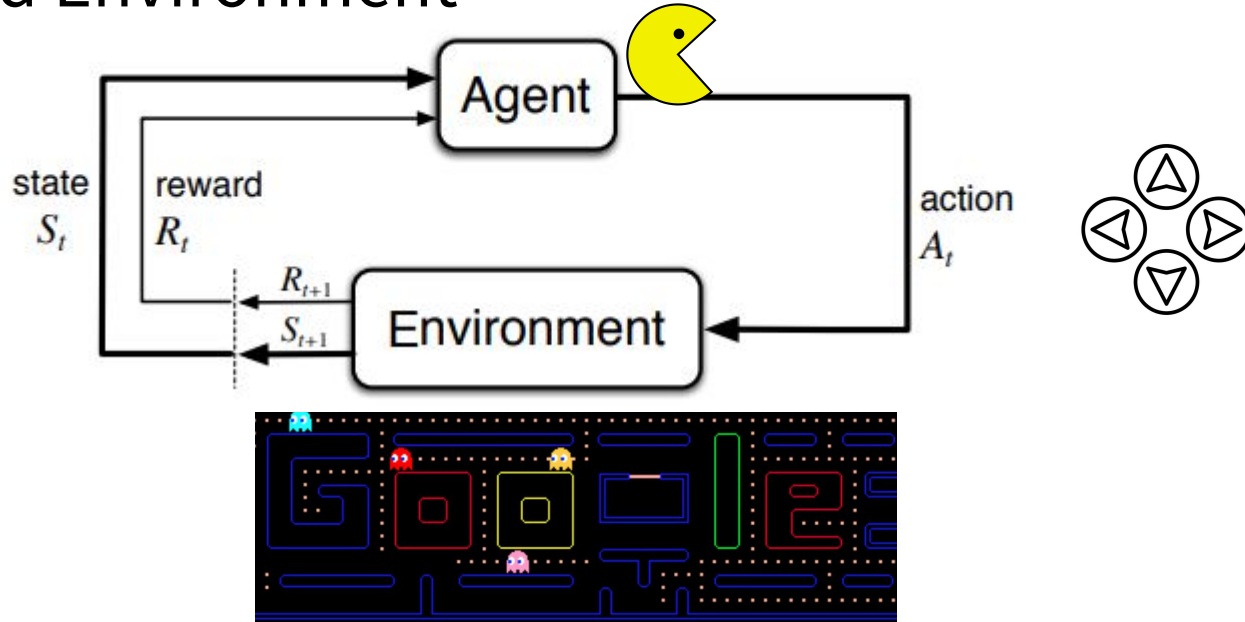
Week [8]: [Deep Q-Learning]

Varsha Kishore, Justin Lovelace, Jinzhou L, John Denero

Logistics

- Will release midterm grades soonish ...
- Releasing a sign-up sheet for notes later today
- Project presentations on last two days of classes
 - Attendance mandatory on both: You present or you grade

Agent and Environment



Agent:

- Perceives environment.
- Makes decisions.
- Aims to maximize reward.

Environment:

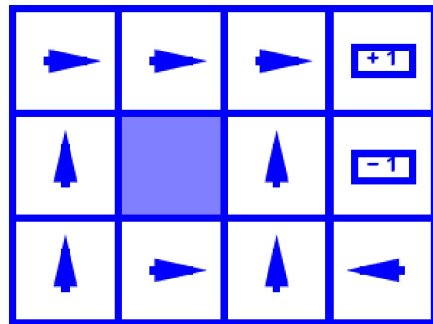
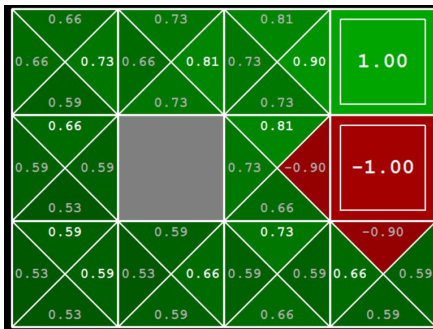
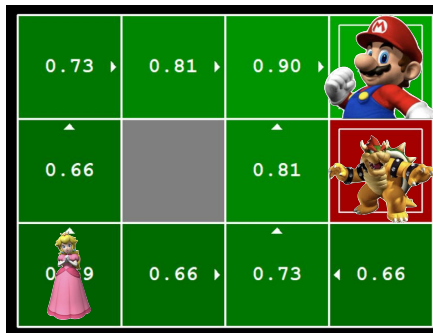
- The external context in which an agent operates and interacts with
- Provides feedback to agent

Markov Decision Process (MDP)

- MDPs provide a framework for modeling sequential decision-making problems.
- An MDP is defined by a tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$:
 - $\mathcal{S}$: Set of states representing the environment.
 - $\mathcal{A}$: Set of actions the agent can take.
 - $\mathcal{P}$: Transition probability function, $\mathcal{P}(s'|s, a)$.
 - $\mathcal{R}$: Reward function, $\mathcal{R}(s_t, a_t, s'_t)$
 - γ : Discount factor, $\gamma \in [0, 1]$.

Optimal Values

- Why? Optimal values define optimal policies!
- Define the value of a state s :
 $V^*(s)$ = expected reward starting in s and acting optimally
- Define the Q-value of (s,a) :
 $Q^*(s,a)$ = expected reward starting in s , taking action a and thereafter acting optimally
- Define the optimal policy:
 $\pi^*(s)$ = optimal action from state s



Recap: Value Iteration

- Idea:

- Start with $V_0^*(s) = 0$
- Given V_i^* , calculate the values for all states at next step:

$$V_{i+1}(s) \leftarrow \max_a \sum_{s'} \mathcal{P}(s'|s, a) [R(s, a, s') + \gamma V_i(s')]$$

- This is called a **value update** or **Bellman update**
 - Repeat until convergence
- Theorem: will converge to unique optimal values
 - Basic idea: approximations get refined towards optimal values
 - Policy may converge long before values do

Limitations of Model Based Learning (e.g. Value Iteration)

Requires full knowledge of the environment (or accurate simulator)

- Needs to know transition probability $P(s' | s, a)$ and Reward function $R(s, a, s')$
- Often unknown in practice

Computationally Expensive

- Updates **all states** for **all actions**
- Doesn't work if state space is too large
- Often we don't need an accurate Value function, **we just want a policy**

Limitations of Model-Based Learning (Value Iteration)

Environmental Knowledge

Requires full knowledge of the environment or an accurate simulator.

- Needs transition probability $P(s' | s, a)$ and Reward function $R(s, a, s')$
- These are often **unknown in practice**

Computational Expense

Processing requirements scale poorly with complexity.

- Updates **all states** for **all actions**
- Fails when state space is too large
- Often, an accurate Value function is unnecessary—**we just want a policy**

Q-Learning (Model-Free)

Core Concept

- Learn optimal behavior **directly from experience**, without an environment model.
- Iterative Update:** Agent moves and updates the Q-function at each step (analogous to SGD).

Update Rule & Convergence

$$Q(s, a) \leftarrow (1 - \alpha) Q(s, a) + \alpha \left[R(s_t, a_t, s'_t) + \gamma \max_{a'} Q(s', a') \right]$$

Eventually converges to the optimal Q^* -function:

$$Q^*(s, a) = R(s_t, a_t, s'_t) + \gamma \sum_{s' \in \mathcal{S}} P(s' \mid s, a) \max_{a' \in \mathcal{A}} Q^*(s', a')$$

How should the agent move? ϵ -Greedy Policy

Simple solution to balance **exploration** and **exploitation**: ϵ -greedy policy

$1 - \epsilon$

Choose the **optimal action**

According to learned Q-values

$$\max_a Q(s, a)$$

ϵ

Choose a **random action**

Ensures environment exploration

- The ϵ -greedy policy ensures that the agent **explores** the environment while still **exploiting** the learned knowledge.
- Despite its simplicity, ϵ -greedy is still **widely used in practice** and often yields good results.

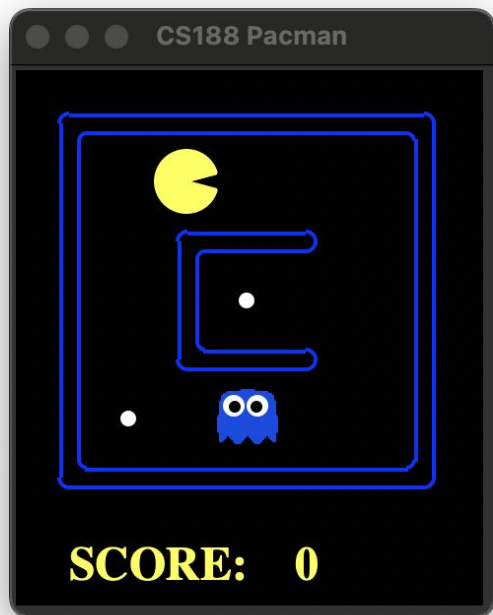
Q-Learning

- 1: Initialize $Q(s, a)$, $\forall s \in \mathcal{S}$, $a \in \mathcal{A}(s)$, and $Q(\text{terminal}, \cdot) = 0$
- 2: repeat (for each episode)
 - 3: Initialize s
 - 4: repeat (for each step of episode)
 - 5: Choose a from s using Q (ϵ -greedy)
 - 6: Take action a , observe R, s'
 - 7: $Q(s, a) \leftarrow (1 - \alpha)Q(s, a) + \alpha \left[R + \gamma \max_{a'} Q(s', a') \right]$
 - 8: $s \leftarrow s'$
 - 9: until s is terminal

Deep Q Learning

Quiz: How many possible states are in this 5x5 and 5x6 games?

(You can ignore the score.)



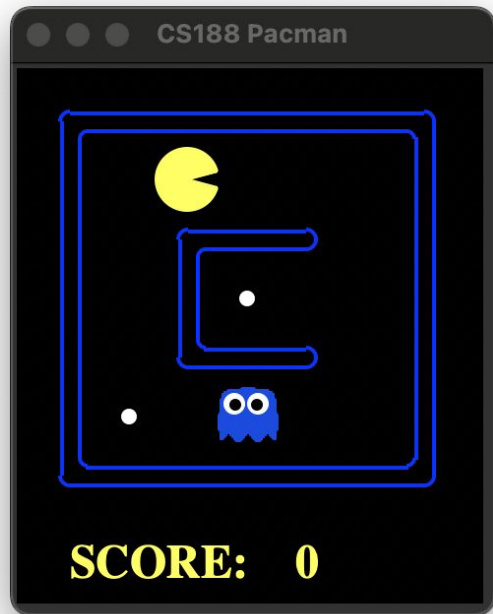
How many unique stats exist in the full pacman game?

- 244 (Pac-Man positions)
- × 4 (Pac-Man directions)
- × 244^4 (ghost positions)
- × 4^4 (ghost directions)
- × 3^4 (ghost modes)
- × 2^{240} (dot states)
- × 2^4 (power pellet states)
- × 2 (fruit presence)
- × Various timer states= A LOT!



Feature Space instead of Lookup Table

Raw “pixel” space



Feature Space
Of (location, action)

$f(s,a)=\{$
(x,y) position of Pacman
<your features here>
 $\}$

$$\mathbf{x} = f(s, a)$$

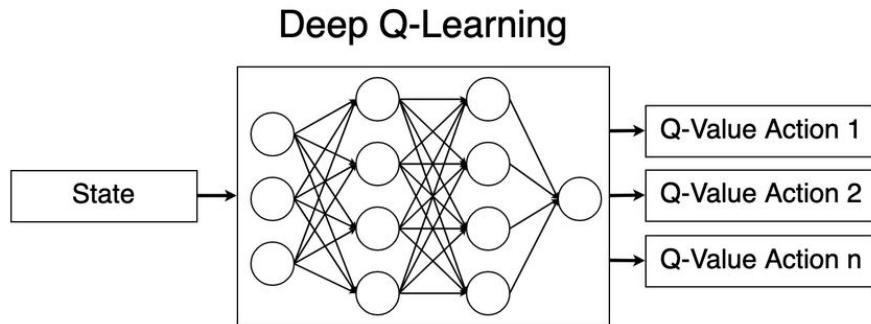
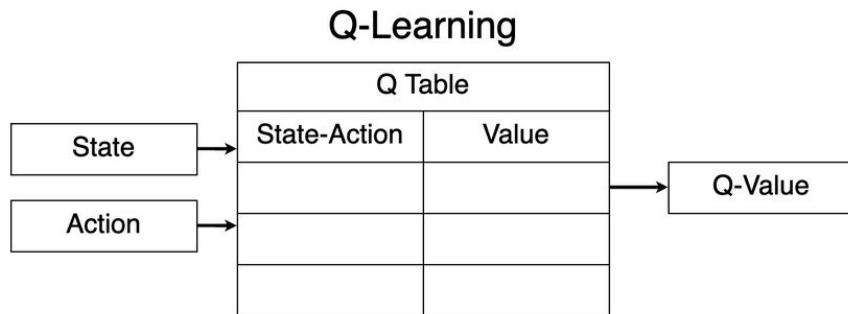
Q-Learning

E.g.
Linear regression
To predict reward

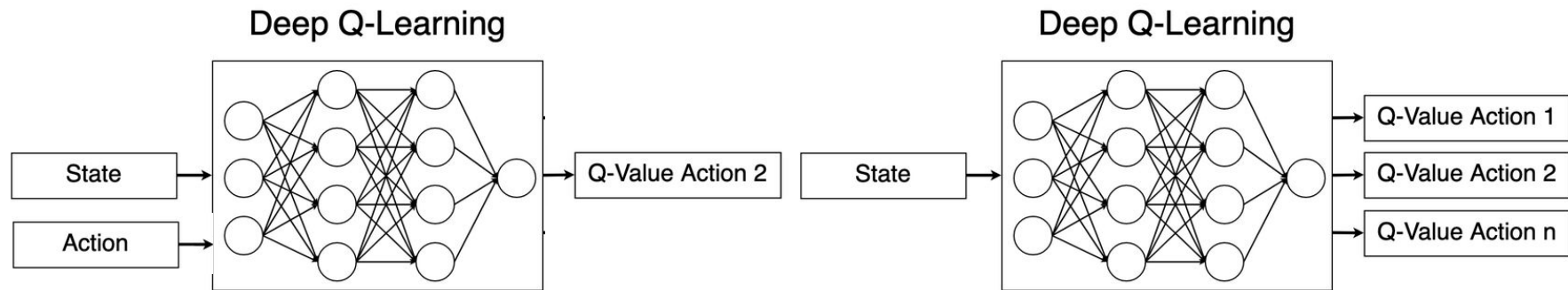
$$Q(s, a) \approx \mathbf{w}^T \mathbf{x}$$

Bonus question: How can you encode the original state space as a feature vector?

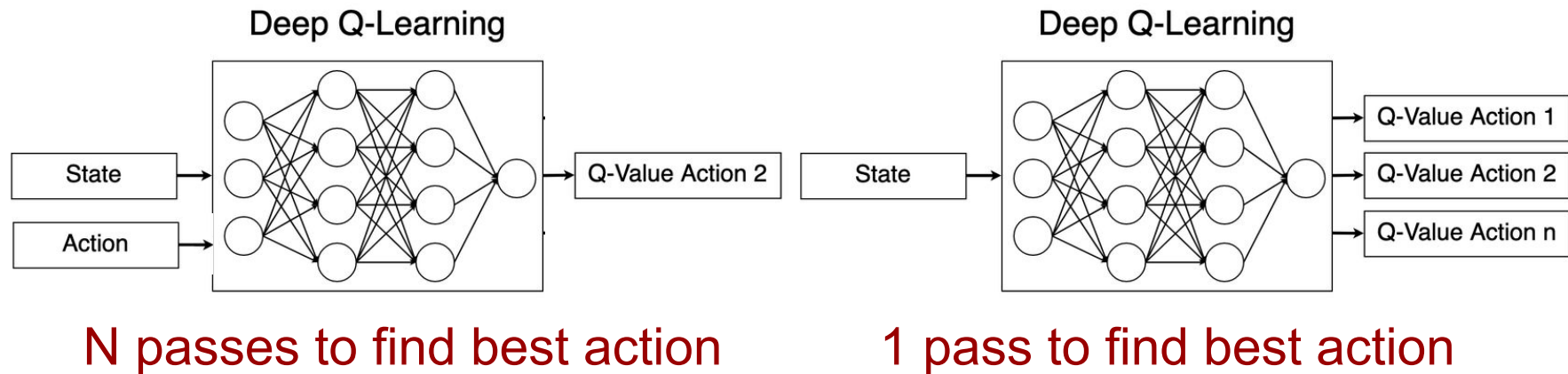
How can we use deep learning to **learn** the feature representation?



Discussion: Which do we prefer?

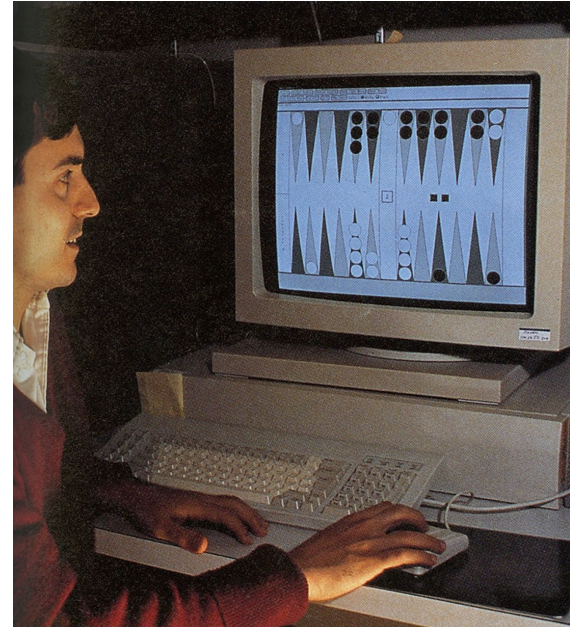


Discussion: Which do we prefer?

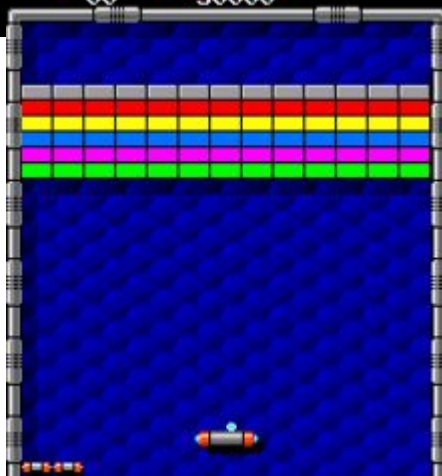
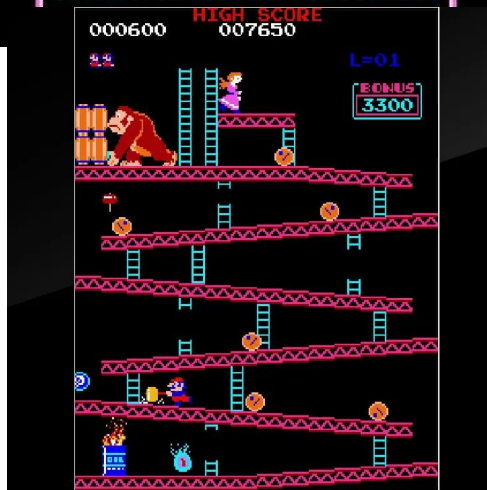


TD-Gammon (1992)

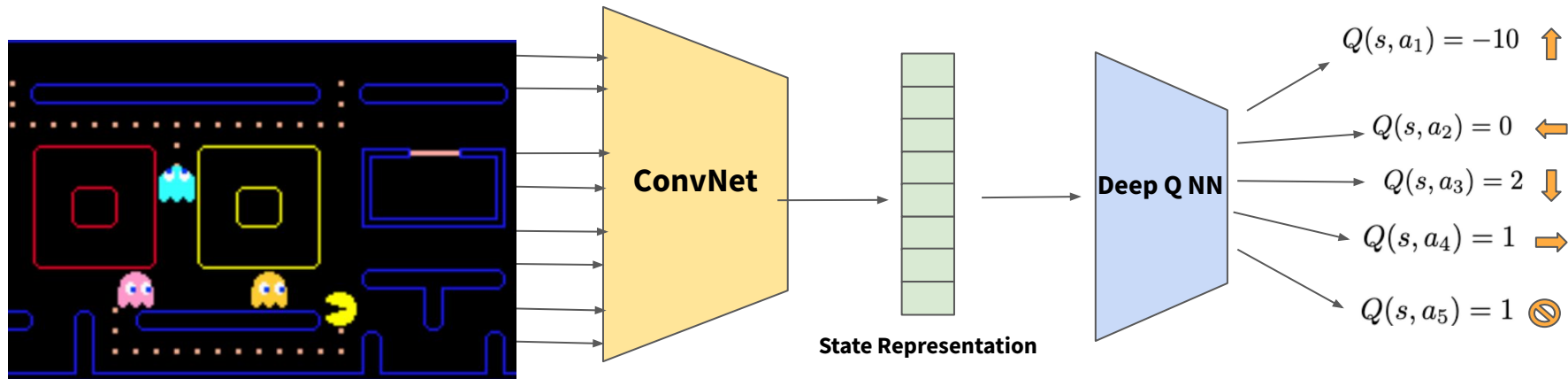
Gerald Tesauro at IBM Research trained a neural network to play Backgammon at world-class level.



Atari Revolution (Deep Mind 2014)



Play Pacman with DQN



Temporal Limitations

Which way are the pacman and the ghosts moving?



Temporal Limitations

Which way are the pacman and the ghosts moving?

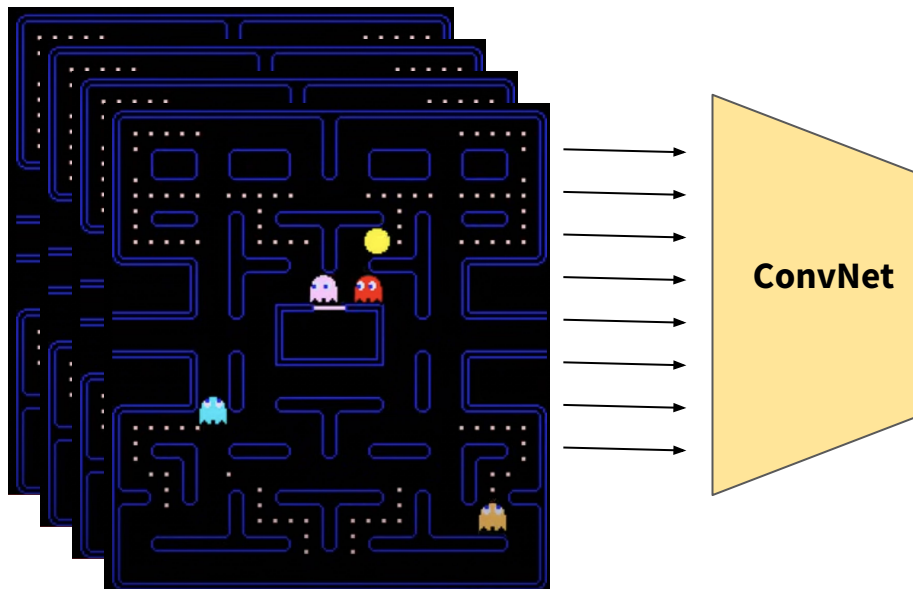


t



$t + 1$

We feed in multiple



Training DQN

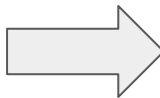
Naive Approach

Agent performs
action

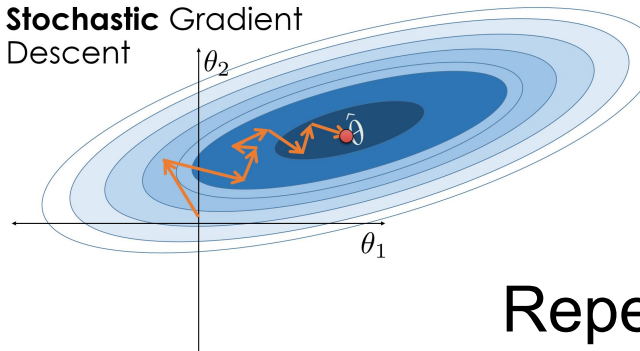


$$(s_t, a_t, r_t, s'_t)$$

Gradient step



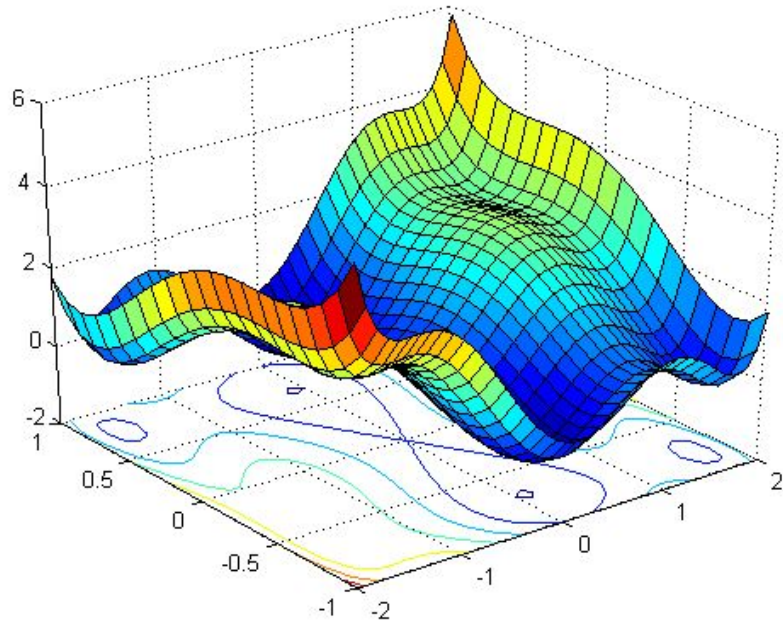
Stochastic Gradient
Descent



Repeat

Problem: High Autocorrelation

Data is not i.i.d

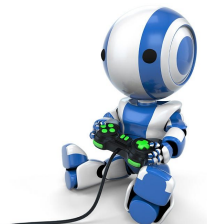


High Correlation between

$$(s_t, a_t, r_t, s'_t) \text{ \& } (s_{t+1}, a_{t+1}, r_{t+1}, s'_{t+1})$$

Replay memory technique: store agent's experiences

Agent performs
action



$$(s_t, a_t, r_t, s'_t)$$

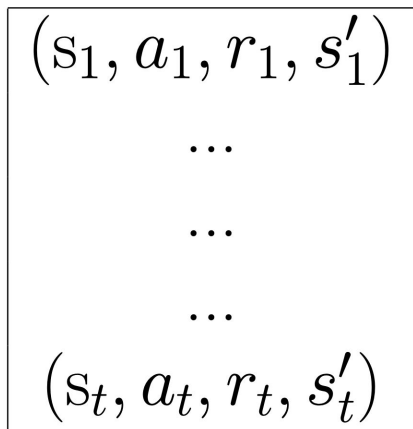


Stored in replay buffer

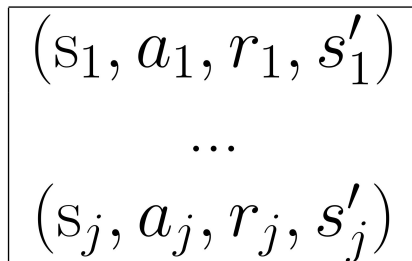
(s_1, a_1, r_1, s'_1)
...
...
...
(s_t, a_t, r_t, s'_t)

Experience Replay

Store Transitions

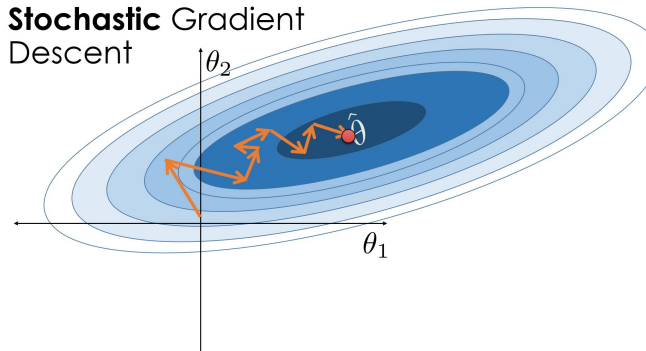


Sample Minibatch



Training

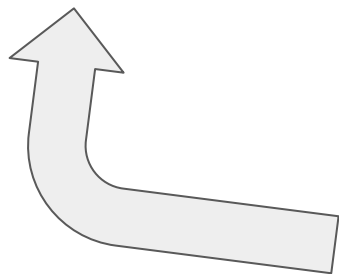
Stochastic Gradient Descent



- Efficient use of training data
- Avoid forgetting previous experiences
- Remove correlations

How do we compute loss?

$$||y - Q(s, a)||_2^2$$



"True" Q value

Temporal Difference (TD) Learning

- Consider the Bellman update

$$Q(s, a) \leftarrow (1 - \alpha)Q(s, a) + \alpha \left[\mathcal{R}(s, a) + \gamma \max_{a'} Q(s', a') \right]$$

- When does this converge?

$$Q(s, a) = \mathcal{R}(s, a) + \gamma \max_{a'} Q(s', a')$$

Discussion: Why might issues arise with this loss function?

$$\left\| \underbrace{r + \gamma \max_a Q(s', a)}_{\text{Temporal Difference (TD) Target}} - \underbrace{Q(s, a)}_{\text{Prediction}} \right\|_2^2$$

Loss is a function of our estimated Q value!

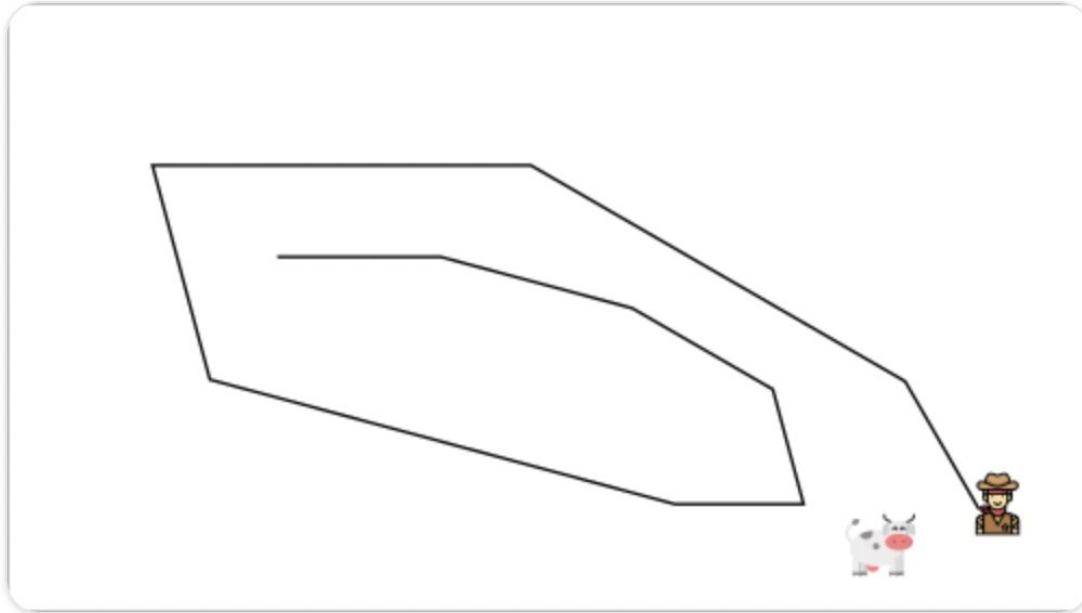
Chasing a moving target!

When $Q(s, a)$ changes, so does y



Chasing a moving target!

When $Q(s, a)$ changes, so does y



Solution: Fix the target to stabilize training

$$\|r + \gamma \max_a \boxed{\overset{\text{lock}}{Q}}(s', a) - Q(s, a)\|_2^2$$

Temporal Difference (TD) Target

- Use a frozen network to compute the target
- Update the frozen network periodically

Deep Q-learning

- Sampling
 - Perform actions and store the observed experience tuples in a replay memory
 - Tuples are of the form (s_t, a_t, r_t, s'_t)
- Training
 - Select a batch from the replay buffer
 - Update neural network based on the batch

Algorithm 1 Deep Q-Learning with Replay Memory and Target Network

```

1: Initialize replay memory  $D$  to capacity  $N$ 
2: Initialize action-value function  $Q$  with random weights  $\theta$ 
3: Initialize target action-value function  $\hat{Q}$  with weights  $\theta^- = \theta$ 
4: for episode = 1,  $M$  do
5:   Initialize state  $s_1$ 
6:   for  $t = 1, T$  do
7:     With probability  $\epsilon$  select a random action  $a_t$ 
8:     otherwise select  $a_t = \arg \max_a Q(s_t, a; \theta)$ 
9:     Execute action  $a_t$  in simulator and observe reward  $r_t$  and new state  $s_{t+1}$ 
10:    Store transition  $(s_t, a_t, r_t, s_{t+1})$  in  $D$ 
11:  end for
12:  Sample random minibatch of transitions  $(s_j, a_j, r_j, s_{j+1})$  from  $D$ 
13:  for each transition  $(s_j, a_j, r_j, s_{j+1})$  in the minibatch do
14:    if episode terminates at step  $j + 1$  then
15:      Set  $y_j = r_j$ 
16:    else
17:      Set  $y_j = r_j + \gamma \max_{a'} \hat{Q}(s_{j+1}, a'; \theta^-)$ 
18:    end if
19:  end for
20:  Perform a gradient descent step on  $\frac{1}{|B|} \sum_j (y_j - Q(s_j, a_j; \theta))^2$  with respect to  $\theta$ 
21:   $\theta^- = \tau \theta + (1 - \tau) \theta^-$ 
22: end for

```

Summary of Models

Q Learning

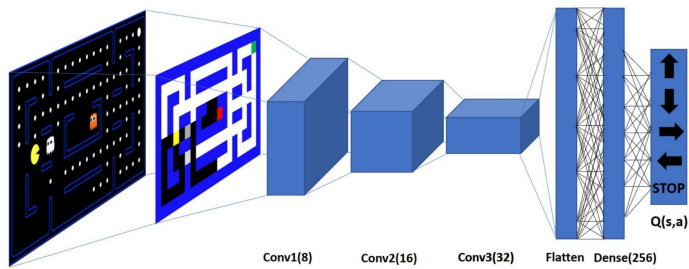
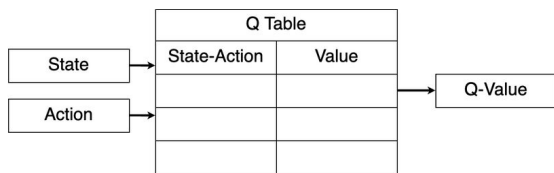
Basic reinforcement learning algorithm

Tabular storage of q-values

Vanilla DQN

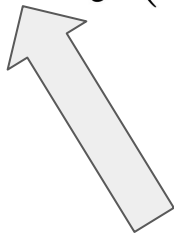
Use ConvNet to represent environment

Neural net for Q-values



Overestimation in DQN

$$y = r + \max_a Q(s', a; \theta^-)$$



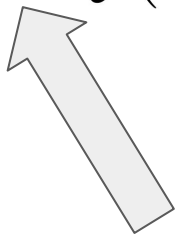
Taking the max of noisy
random variables

Overestimation Intuition

- There are 300 people with the same weight: 150 pounds
- There is a weight scale that measures with an error of ± 5
- Suppose someone (who doesn't know the original weights) wants to estimate the max weight
 - Method 1:
 - sample n people and weigh them to obtain a list of weights $x_1, \dots, x_n$
 - Output $\max(x_1, \dots, x_n)$
 - Method 2:
 - Weigh each of the n people twice to obtain $x_1, \dots, x_n$ and $x'_1, \dots, x'_n$
 - Find $i = \operatorname{argmax}(x_1, \dots, x_n)$ and then output x'_i

Overestimation in DQN

$$y = r + \max_a Q(s', a; \theta^-)$$

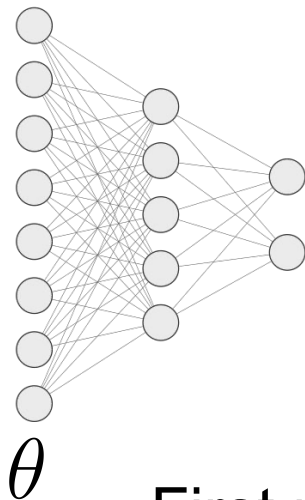


Taking the max of noisy
random variables

Idea: Use different
networks to select action
& evaluate action

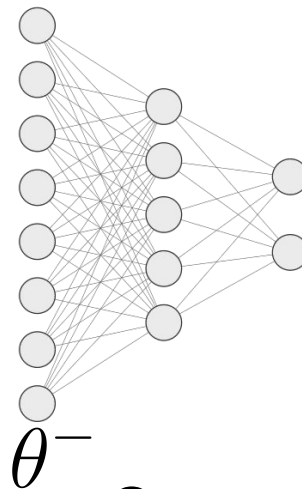
Double DQN

$$a^* = \arg \max_a Q(s', a; \theta)$$



First network
selects best action

$$y = r + \gamma Q(s', a^*; \theta^-)$$



Second network
evaluates action

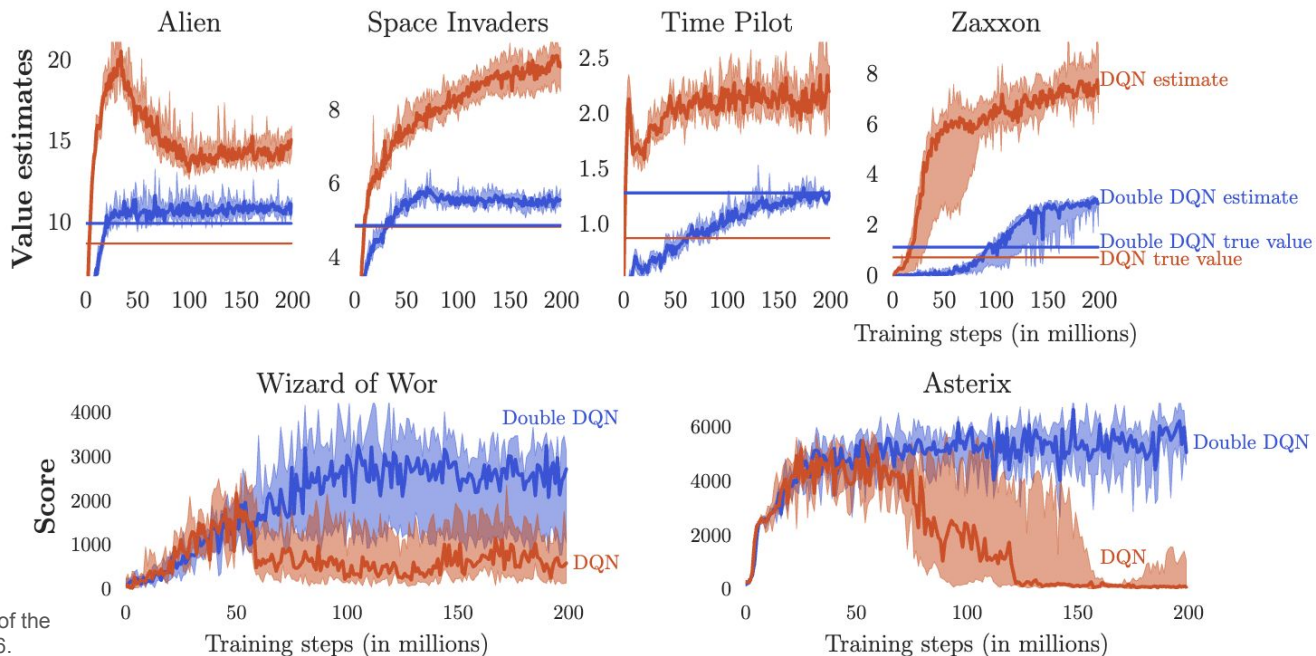
Discuss: What's the difference between Fixed Q-Targets & Double DQN?

Algorithm 2 Double Deep Q-Learning

```
1: Initialize replay memory  $D$  to capacity  $N$ 
2: Initialize action-value function  $Q$  with random weights  $\theta$ 
3: Initialize target action-value function  $\hat{Q}$  with weights  $\theta^- = \theta$ 
4: for episode = 1,  $M$  do
5:   Initialize state  $s_1$ 
6:   for  $t = 1, T$  do
7:     With probability  $\epsilon$  select a random action  $a_t$ 
8:     otherwise select  $a_t = \arg \max_a Q(s_t, a; \theta)$ 
9:     Execute action  $a_t$  in simulator and observe reward  $r_t$  and new state  $s_{t+1}$ 
10:    Store transition  $(s_t, a_t, r_t, s_{t+1})$  in  $D$ 
11:  end for
12:  Sample random minibatch of transitions  $(s_j, a_j, r_j, s_{j+1})$  from  $D$ 
13:  for each transition  $(s_j, a_j, r_j, s_{j+1})$  in the minibatch do
14:    if episode terminates at step  $j + 1$  then
15:      Set  $y_j = r_j$ 
16:    else
17:       $a^* = \max_{a'} Q(s_{j+1}, a'; \theta)$ 
18:      Set  $y_j = r_j + \gamma \hat{Q}(s_{j+1}, a^*; \theta^-)$ 
19:    end if
20:  end for
21:  Perform a gradient descent step on  $\frac{1}{|B|} \sum_j (y_j - Q(s_j, a_j; \theta))^2$  with respect to  $\theta$ 
22:   $\theta^- = \tau \theta + (1 - \tau) \theta^-$ 
23: end for
```

Impact of Double DQN

- Double DQN reduces over-estimation
- Stabilizes training



Summary of Models

Q Learning	Vanilla DQN	Double DQN
Basic reinforcement learning algorithm	Use ConvNet to represent environment	Fixes the overestimation problem by using different networks to find and evaluate the best action
Tabular storage of q-values	Neural net for Q-values	

Review

- Tabular q-learning does not scale well when there are a lot of states
- Deep q-learning uses neural networks and is able to handle problems with large state-action spaces
- Issues with vanilla deep q-learning:
 - Correlation between subsequent time steps
 - Moving target
- Vanilla deep q-learning can be further improved by fixing the overestimation problem and using double deep q-learning